RISC-V Single Cycle Processor

GitHub Repository

Jr. ECE Student

Ahmed Samier

Al-Azhar University

LinkedIn Profile

Contents:

1. Introduction: ***3***

2. Objective ***3***

3. Architecture Design ***4***

4. ALU Truth Table ***6***

5. Main Decoder Truth Table ***6***

6. Instruction Set ***6***

7. ALU Decoder Truth Table ***7***

8. Integration of Blocks ***7***

9. Datapath of Load Word, Store Word, Branch If Equal ***8***

- Load Word 8
- Store Word 9
- Branch Equal 10

10. Controlling PCSrc in Branch Instructions ***10***

- beq (Branch if equal) 11
- bnq (Branch if not equal) 11
- blt (Branch if less than) 11

11. Simulation Results: ***12***

- Datapath Signals Waveform 12
- Control Unit Signals Waveform 12
- Testbench Result 12

12. Project Repository & Contact: ***13***

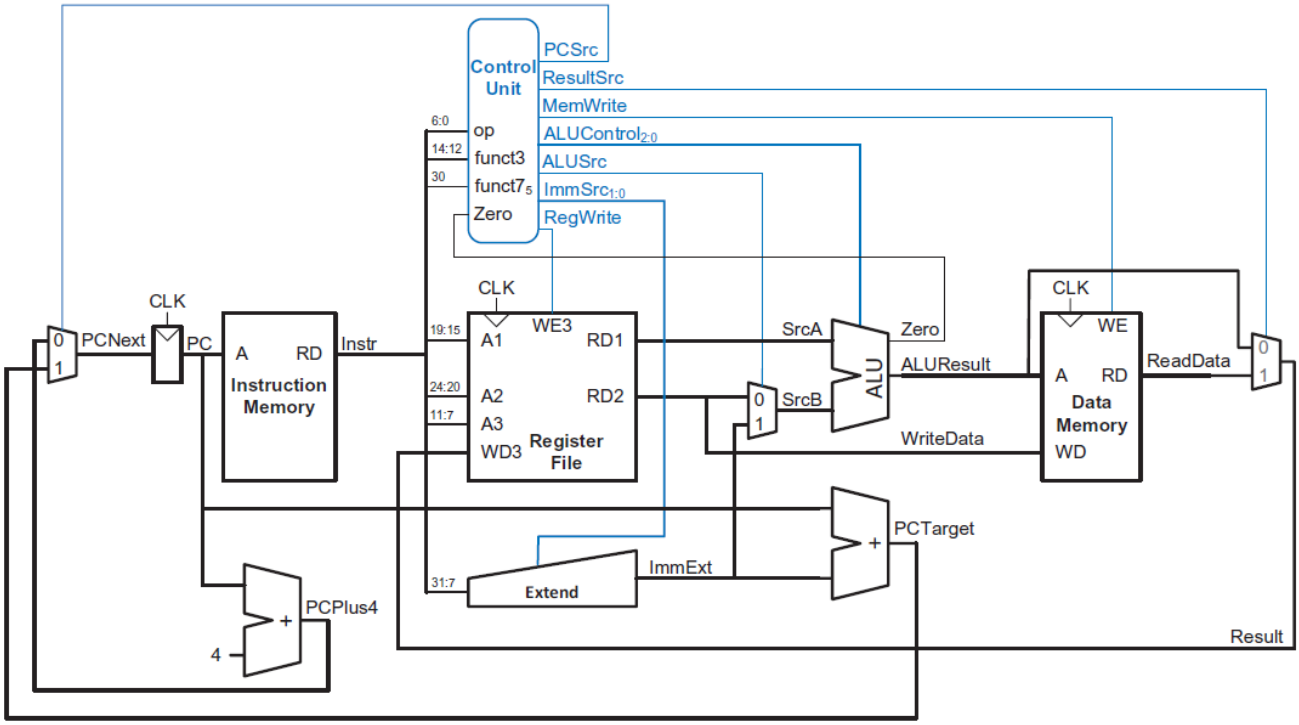
1. Introduction:

This project focuses on designing and implementing a 32-bit single-cycle RISC-V processor based on the **Harvard architecture**. In a single-cycle design, every instruction is completed within one clock cycle, covering all essential stages of execution**: instruction fetch, decode, execution, write-back**, and **program counter update**. The Harvard architecture separates instruction memory from data memory, allowing clearer organization and improved conceptual understanding of processor operation.

2. Objective:

The objective of this project is to develop RTL Verilog modules for the core components of the processor — such as the **register file**, **instruction memory**, **ALU**, and **control unit** — and integrate them into a complete top-level processor design. The final system is then simulated to **verify** correct functionality and instruction execution.

3. Architecture Design:



The processor executes each instruction in one clock cycle by moving data through a fixed sequence of hardware stages. Based on the shown architecture, the instruction flows through fetch, decode, execute, memory access, and write-back in a continuous path within the same cycle.

At the beginning of the cycle, the Program Counter (**PC**) holds the address of the current instruction. This address is sent to the instruction memory, which returns the instruction bits. At the same time, an adder calculates **PC + 4**, preparing the address of the next sequential instruction.

The fetched instruction is then decoded. The control unit reads the opcode and function fields and generates control signals that determine how the **datapath** behaves. The **register file** reads two source registers specified by the instruction, while the immediate generator extends constant values when required.

Next comes the execution stage. The **ALU** receives its operands from the register file or the **immediate value** depending on the instruction type. It performs arithmetic or logical operations and produces a result. For branch instructions, the ALU also generates a **Zero signal** used by the control unit to decide whether a branch should be taken.

If the instruction requires memory access, the ALU result is used as an address for the data memory. For load instructions, data is read from memory. For store instructions, data from the register file is written to memory.

Finally, in the write-back stage, either the ALU result or memory data is written back into the destination register. A multiplexer selects the correct value based on the control signals. At the end of the cycle, the PC is updated. It either moves to PC + 4 or to a branch target address computed earlier. The next cycle then begins with the new instruction.

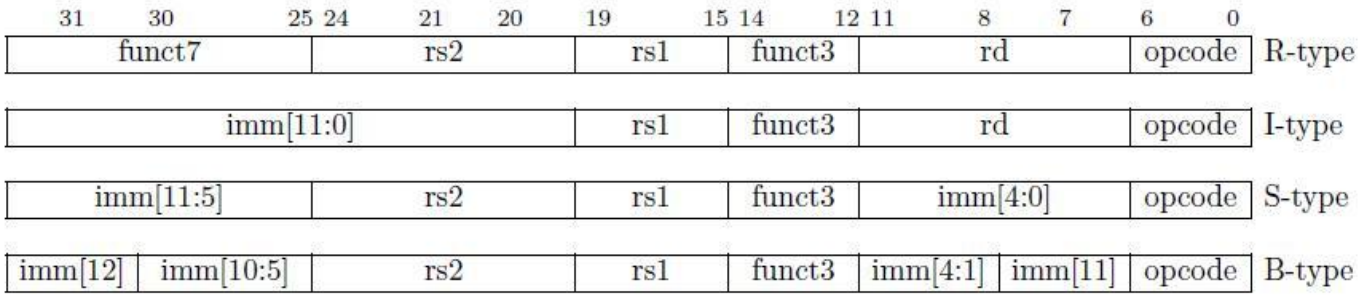
4. ALU Truth Table:

ALUControl	Operation
000	SrcA + SrcB
001	SrcA << SrcB
010	SrcA - SrcB
100	SrcA **xor** SrcB
101	SrcA >> SrcB
110	SrcA **or** SrcB
111	SrcA **and** SrcB

5. Main Decoder Truth Table:

Instruction	Operation	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
lw	0000011	1	00	1	0	1	0	00
sw	0100011	0	01	1	1	x	0	00
R-Type	0110011	1	xx	0	0	0	0	10
beq	1100011	0	10	0	0	x	1	01
addi	0010011	1	00	1	0	0	0	10

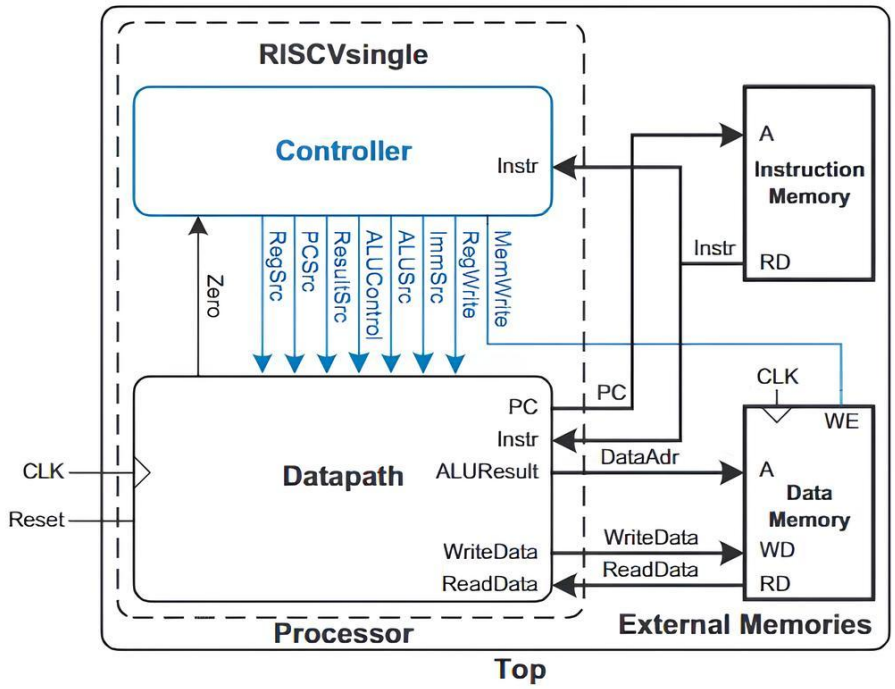
6. Instruction Set:



7. ALU Decoder Truth Table:

ALUOp	funct3	{op5, funct7}	ALUControl	Instruction
00	xxx	xx	000	lw, sw (add)
01	000	xx	010	beq (sub)
	001	xx	010	bnq (sub)
	100	xx	010	blt (sub)
10	000	00, 01, 10	000	add
	000	11	010	sub
	001	xx	001	SHL (shift left)
	100	xx	100	XOR
	101	xx	101	SHR (shift right)
	110	xx	110	OR
	111	xx	111	AND

8. Integration of Blocks:



9. Datapath of Load Word, Store Word, Branch If Equal:

- Load Word:

Step 1: The type of instruction used will determine the path for data to flow. To explain in simpler ways, the data will follow a path from the program counter at an initial address. It will give the instruction in machine code language after being fed into the instruction Memory.

Step 2: The opcode and instruction used will excute path in a certain direction. To know which path to choose, the multiplexer is been placed.

Step 3:The sourse register from instruction set is fed into the register file at source operand A1(for single source register) which now gives us SrcA. For SrcB, add the extend file which will convert the small bits to 32 bit wide.

Note: If two single source register are used then both A1 and A2 source operand are used.

Step 4: The SrcA and SrcB will be computed in ALU Logice where it comprise of Main Decoder and ALU Decoder which are controlled by ALU Controller.

Step 5: The ALUResult is fed into the Data memory and where the Result is added back to the Register File of WD3. RegWrite is added as a control signal to ALU Controller. Now, we need the next instruction to pass on, the adder is used which gives output as PCPlus4 which addes the previous input with four.

- Store Word:

The whole structure of store word is almost same like the load word. The changes to be made are:

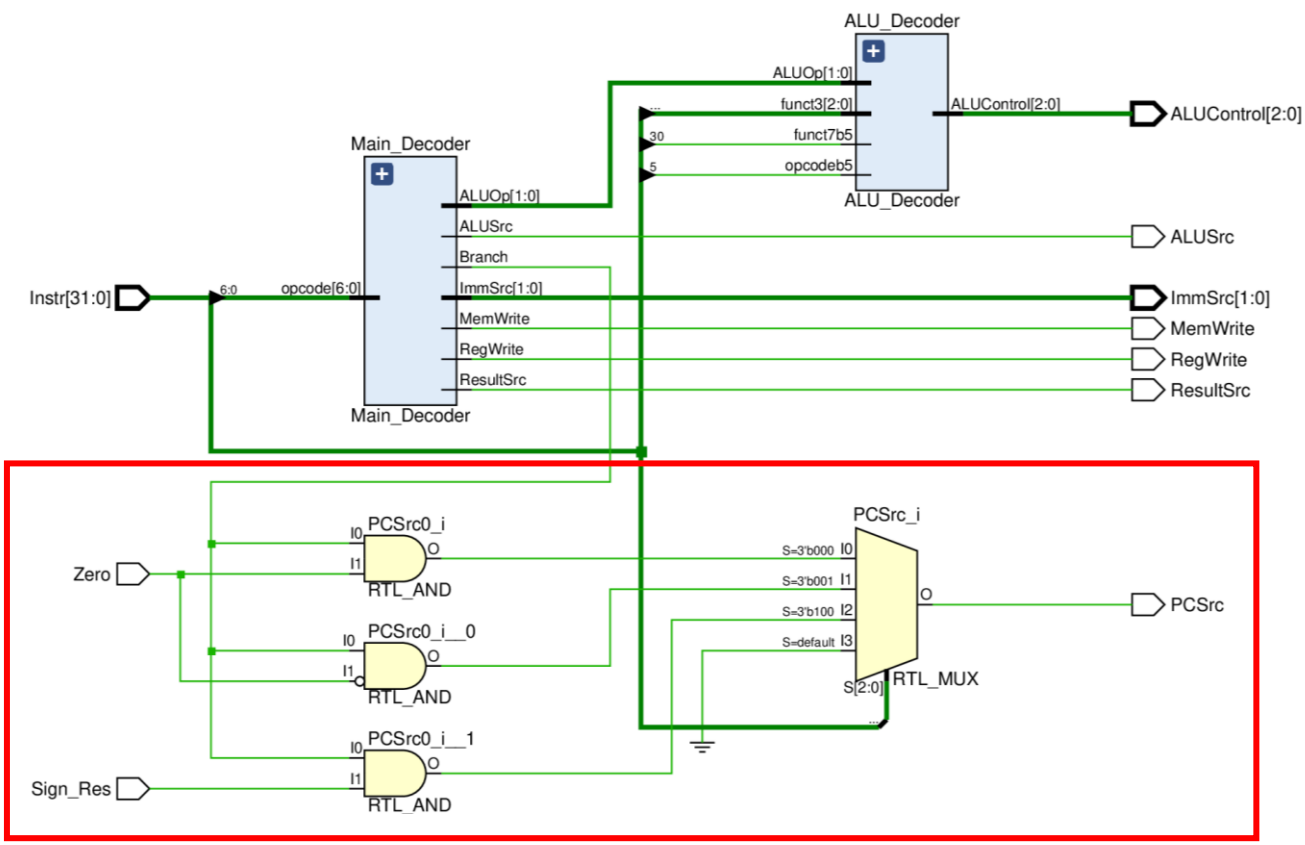
- Two single source register is used.
- ImmSrc from from extend file and MemWrite from the Data memory as a control signal is supplied to the ALU Controller.
- Second Register is fed into the A2 source operand.

- Branch Equal:

Follow the same data path till store word. additionals are:

- Perform substraction operation on SrcA and SrcB, if the result gives us zero then add zero flag to show the results are obtained for beq instructions.
- ResultSrc and RD is disabled.
- Calcualte the Target Address PCTarget = PC input + ImmExt the PCTarget is fed into the Program Counter by using multipler.

10. Controlling PCSrc in Branch Instructions:



- **beq (Branch if equal):**

  The PCSrc signal is set when the Branch control is high and the ALU result indicates equality (Zero = 1).

- **bnq (Branch if not equal):**

  The PCSrc signal is set when the Branch control is high and the ALU result is not zero.

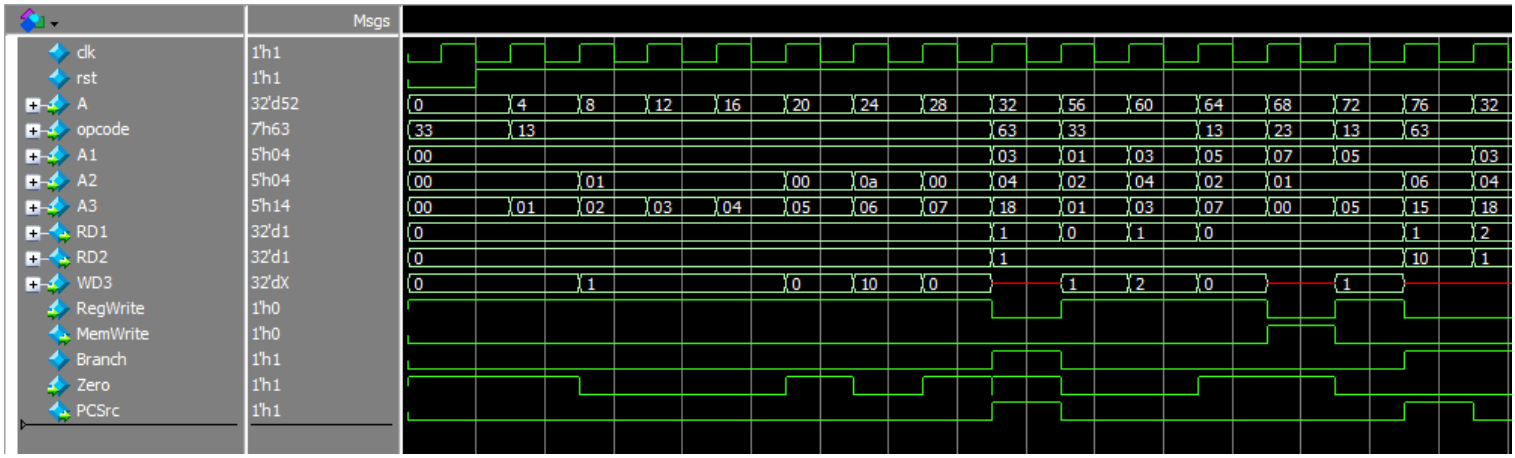
- **blt (Branch if less than):**

  For signed comparison, the signal is set when Branch is high and the MSB of the ALU result indicates a negative outcome (Sign_Res = DataAdr[31]).

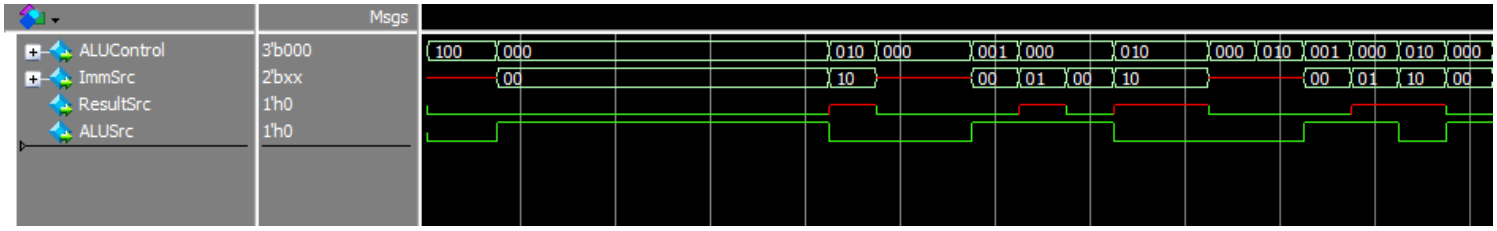
```
always @(*) begin
        case (funct3)
            3'b000: PCSrc = Branch & Zero; //beq
            3'b001: PCSrc = Branch & ~Zero; //bnq
            3'b100: PCSrc = Branch & Sign_Res; //blt
            default: PCSrc = 0;
        endcase
    end
```

11. Simulation Results:

- Datapath Signals Waveform:



- Control Unit Signals Waveform:



- Testbench Result:

# Address:	**0,**	**MemoryWrite:**	**1,**	**WriteData:**	**1**
# Address:	**4,**	**MemoryWrite:**	**1,**	**WriteData:**	**2**
# Address:	**8,**	**MemoryWrite:**	**1,**	**WriteData:**	**3**
# Address:	**12,**	**MemoryWrite:**	**1,**	**WriteData:**	**5**
# Address:	**16,**	**MemoryWrite:**	**1,**	**WriteData:**	**8**
# Address:	**20,**	**MemoryWrite:**	**1,**	**WriteData:**	**13**
# Address:	**24,**	**MemoryWrite:**	**1,**	**WriteData:**	**21**
# Address:	**28,**	**MemoryWrite:**	**1,**	**WriteData:**	**34**
# Address:	**32,**	**MemoryWrite:**	**1,**	**WriteData:**	**55**
# Address:	**36,**	**MemoryWrite:**	**1,**	**WriteData:**	**89**

12. Project Repository & Contact:

All project files, including RTL code, schematics, simulation waveforms, and results, are available in the GitHub repository.

For any questions or inquiries regarding the project, you can contact me at: ahmedsamier920@gmail.com

For discussions, projects, updates, and concept explanations, you can reach me on LinkedIn and YouTube.

Thanks.